LUNAR LANDING: THE FINAL DESCENT

Velocity and 3D Trajectory from a Descent Camera

T+00:23:10. The inertial log of the final descent is corrupted. Only the camera survived: 144 images from the pitch-over to touchdown. Recover the lander's velocity and its path in 3D, using nothing but classical computer vision.

FRAMES	270 to 413	IMAGES	144, consecutive
TOOLKIT	classical CV only	TOTAL	100 marks



T+00:23:10
TOUCHDOWN AHEAD
LANDER: ILLUSTRATION, CNSA/NASA

MISSION PIPELINE

01 FIND WHERE THE IMAGE CHANGES
Gradients
Trackable points

02 TRACK ONE POINT
Lucas-Kanade
One-step flow

03 TRACK FAST MOTION
Iterative LK
SIFT-guided LK

04 RECONSTRUCT THE FLIGHT
Provided flight code
Tracker comparison

ASSIGNMENT 3

MISSION BRIEF

ES666 Assignment 3: Velocity and 3D Trajectory from a Descent Camera

4
TASKS

INCOMING TRANSMISSION · T+00:23:10 PRIORITY: MISSION CRITICAL

The lander has pitched over and is looking straight down at the surface. Its inertial log for the final descent is corrupted: mission control no longer knows how fast it came down, where it drifted, or when it stopped to hover. Only the descent camera survived, **144 consecutive images** from frame 270 to touchdown at frame 413.

Your team has one objective: **recover the lander's velocity and its 3D trajectory from the images alone, using classical computer vision.** No neural networks, no OpenCV, no black-box trackers. Find where the image changes, track one point, track fast motion, and connect what you built to the provided pipeline to reconstruct the flight.

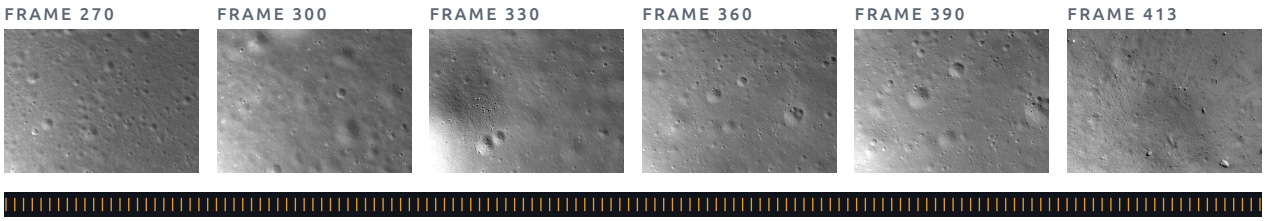


FIGURE 1 The final descent, from the pitch-over to touchdown. Every frame of this part of the landing is yours.

WHAT YOU WILL LEARN

LEARNING OBJECTIVES

By the end of the mission you should be able to:

1. compute finite-difference gradients and connect them to a motion constraint TASK 1
2. pick trackable points from the local structure tensor TASK 1
3. solve Lucas-Kanade as least squares, one step at a time TASK 2
4. make Lucas-Kanade follow large motion, iteratively and with SIFT guesses TASK 3
5. connect your trackers to a provided pipeline to reconstruct the 3D flight TASK 4
6. say what images can and cannot tell you about physical motion REPORT

HOW THE TASKS CONNECT

MISSION ARCHITECTURE

The four missions form one pipeline. Each task produces what the next one needs.

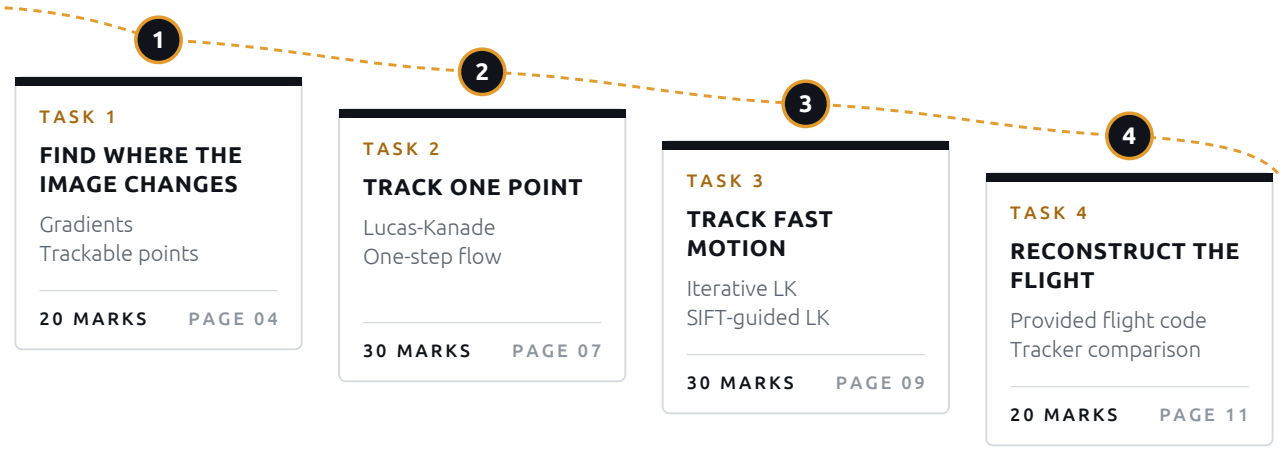


FIGURE 2 Mission roadmap: from gradients to a 3D trajectory.

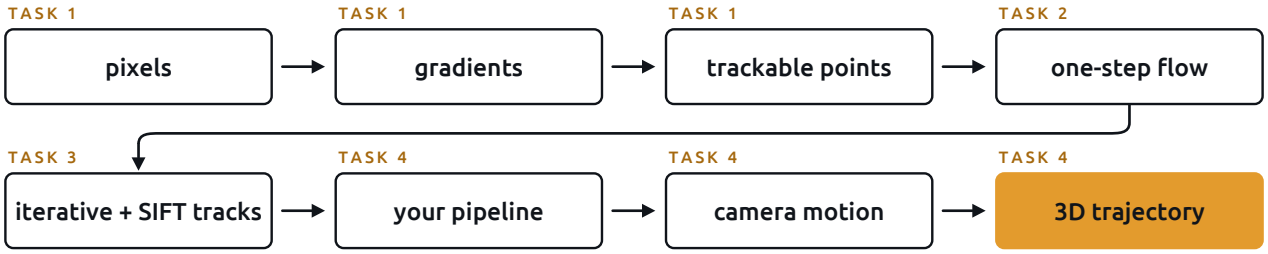


FIGURE 3 The computational chain you will build, from raw pixels to the lander's path.

RULES OF THE TOOLKIT

LIBRARIES

ALLOWED

Python 3, NumPy, Pillow (PIL), Matplotlib and the Python standard library.

NumPy linear algebra such as `np.linalg.lstsq`, `np.linalg.eigvalsh` and `np.linalg.norm` is allowed.

Provided, not for you to write: SIFT (`provided/sift.py`) to start Lucas-Kanade on large motion, and `plotly`, only for the 3D animation.

PROHIBITED

Do not use:

- × OpenCV
- × scikit-image
- × SciPy
- × PyTorch / TensorFlow / JAX / Kornia
- × pretrained models, detectors or descriptors
- × ready-made motion or pose estimation

MISSION DATA

`data/frames/`: frames 270 to 413, 144 consecutive PNG images of 2352 x 1728 pixels from the Chang'e-5 landing camera, taken on 1 December 2020 as the lander looked down at the surface (level 2A data, `moon.bao.ac.cn`; cite it in your report).

`data/mission_constants.json`, released by mission control: the focal length in pixels, the frame interval in seconds and the lander's altitude at one frame.

TASK 01

FIND WHERE THE IMAGE CHANGES

Image gradients, the motion constraint and trackable points

20 MARKS

OBJECTIVE

Learn how motion is written into image intensities: compute image gradients, connect them to motion, and find the points that can be tracked.

STORY · COMMS LOG INCOMING

The navigation officer studies the first frames after the pitch-over.

› "The ground is still there in every frame."
› "Then learn to read it. Every change of brightness is a clue to how we moved."

REQUIRED WORK

1.1 Spatial gradients

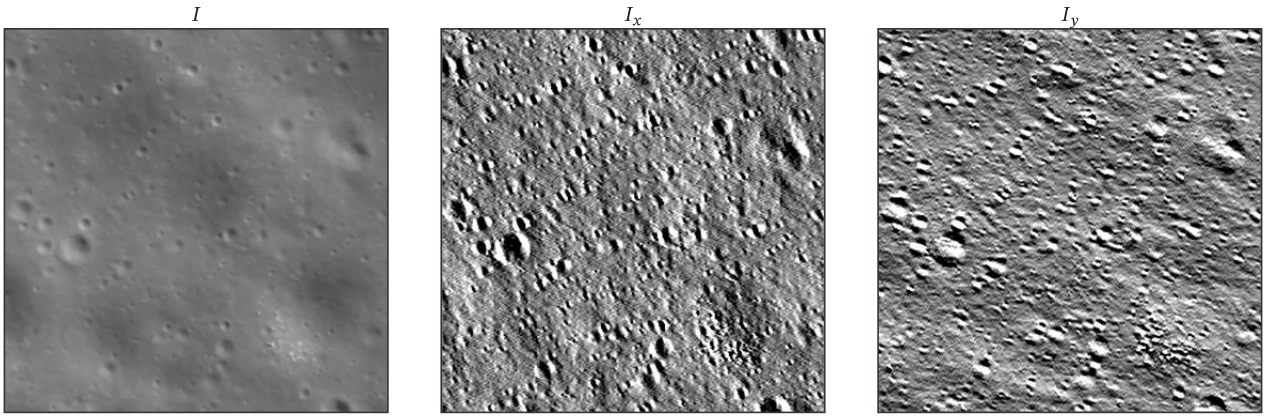
6 marks

Implement, without `np.gradient`:

$$I_x(x,y) = \frac{I(x+1,y) - I(x-1,y)}{2}, \qquad I_y(x,y) = \frac{I(x,y+1) - I(x,y-1)}{2}. \tag{1.1}$$

Show I_x, I_y and the gradient magnitude of frame 276.

ANSWER (a) Where are the gradients strongest, and where are they weakest?



A REAL CROP OF FRAME 276, WITH ITS SPATIAL GRADIENTS GREY = ZERO

FIGURE 4 A real crop of frame 276 and its spatial gradients: grey means zero, and the sign of the change shows as light or dark.

1.2 Temporal change and motion

5 marks

Implement $I_t = I_1 - I_0$. For a small motion (u, v), brightness constancy gives the motion constraint

$$I_x u + I_y v + I_t \approx 0, \tag{1.2}$$

Check it numerically on a synthetic shift of known (u, v).

ANSWER (a) What mean residual does the check report (saved in `constraint_check.json`)? (b) Why can one pixel not give both u and v?

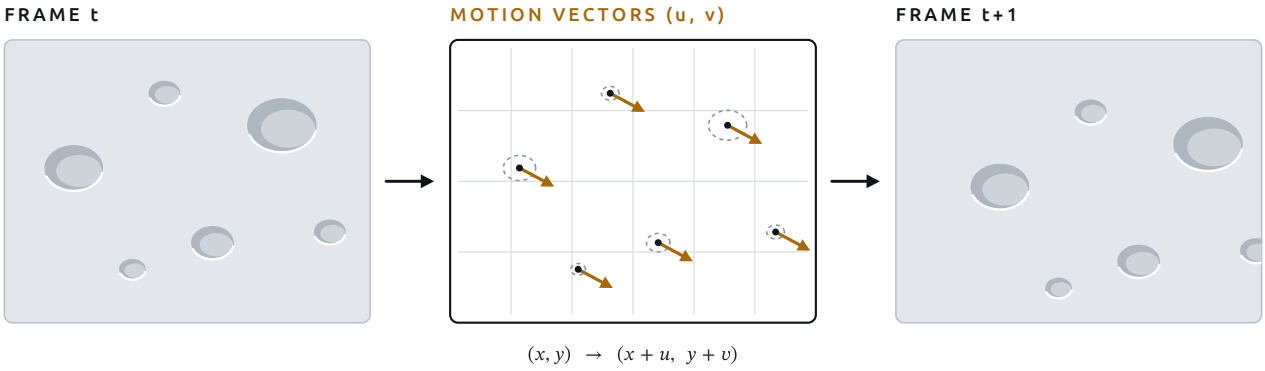


FIGURE 5 A point moves by (u, v) from one frame to the next: the motion constraint links that motion to the gradients and the change in brightness.

1.3 Choosing good points to track

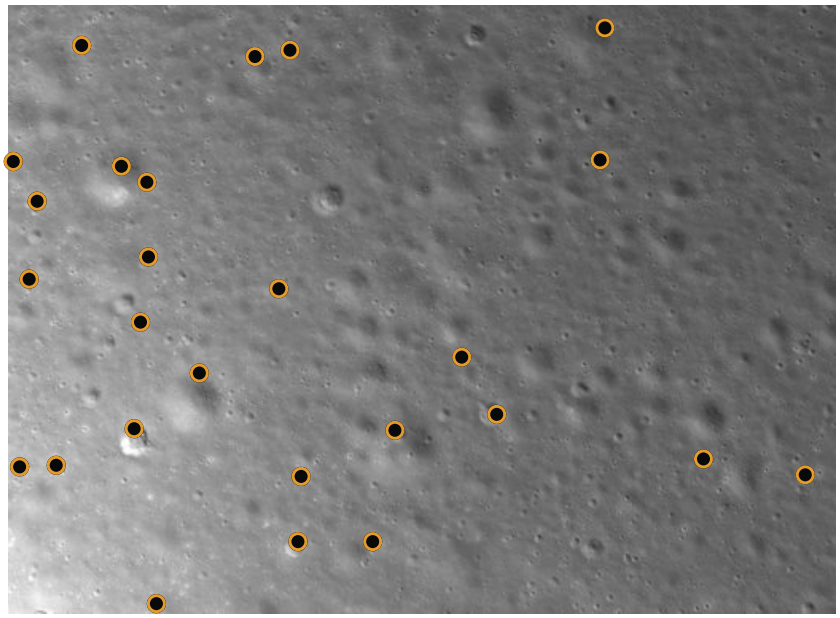
9 marks

For each local window build $G = A^T A$, with one row $[I_x, I_y]$ of A per pixel, and use

$$G = A^T A, \qquad q(x, y) = \min(\lambda_1, \lambda_2) \tag{1.3}$$

as a trackability score. Select strong points with a minimum separation and show them on frame 276. The same matrix G is what Lucas-Kanade solves in Task 2.

ANSWER (a) What does the minimum distance between selected points prevent? (b) What do the eigenvalues of G look like on a flat patch, an edge and a corner? (c) Why does a large q mean a point is easy to track?



26 TRACKABLE POINTS, FRAME 276 (STRONGEST FIRST, SPACED APART)

FIGURE 6 Real trackable points on frame 276, picked strongest first with a minimum separation between them.

STARTER-CODE INTERFACE · TASK1.IPYNB

signatures only: the bodies are yours to write

```
# YOUR CODE cells of task1.ipynb   (no np.gradient)
def spatial_gradients(image): ...
    # ix[1:-1, 1:-1] = (image[1:-1, 2:] - image[1:-1, :-2]) / 2; iy likewise on rows
def temporal_difference(image0, image1): ...
    # image1 - image0, both as float64
def structure_tensor(ix, iy, x, y, window_radius): ...
    # sum Ix*Ix, Ix*Iy, Iy*Iy over the window -> the 2x2 matrix G
def feature_score(G): ...
    # np.linalg.eigvalsh(G)[0], the smaller eigenvalue
def select_features(image, max_points=200, window_radius=3, min_distance=10,
                    threshold=None): ...
    # score every window, sort by q (np.argsort), keep points min_distance apart
```

DELIVERABLES

TASK 1 · 20 MARKS

- ☐ task1.ipynb: your code, the checks run, your answers
- ☐ outputs/task1/gradients.png
- ☐ outputs/task1/constraint.png
- ☐ outputs/task1/constraint_check.json
- ☐ outputs/task1/features.png

MISSION CHECKPOINT

You now have:

images → gradients → the motion constraint → trackable points

TASK 02

TRACK ONE POINT

Lucas-Kanade, one step at a time

30 MARKS

OBJECTIVE

Estimate the motion of one point with Lucas-Kanade, and find out when a single step is enough.

STORY · COMMS LOG INCOMING

The navigation officer picks one rock and tries to follow it.

› "Pick one rock and follow it."
› "One step at a time."

REQUIRED WORK

2.1 Lucas-Kanade as least squares — 12 marks

Assume constant motion in a local window and solve, with NumPy linear algebra,

$$A\begin{bmatrix}u\\v\end{bmatrix} \approx b, \qquad b = -\begin{bmatrix}I_{t1}\\ \vdots \\ I_{tN}\end{bmatrix}. \tag{2.1}$$

ANSWER (a) Write the least-squares solution for (u, v) using $G = A^TA$. (b) When can it not be solved? Relate this to q from Task 1.3.

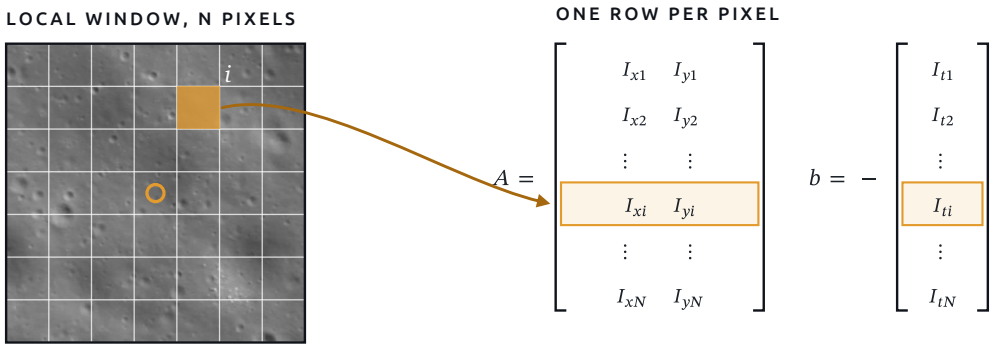


FIGURE 7 Every pixel of the local window contributes one row of A and one entry of b; the motion (u, v) is shared by the whole window.

2.2 Estimate one-step motion — 9 marks

Sample the gradient and temporal-difference windows at a point (with the provided bilinear sampling) and solve once. The provided tests and the Task 2.3 sweep check it on synthetic sub-pixel shifts.

ANSWER (a) How large is the error of `one_step_flow` on sub-pixel shifts? See the smallest shifts of the Task 2.3 sweep (`error_sweep.json`).

2.3 When one step is not enough

9 marks

Run the provided sweep of synthetic shifts from 0.1 to 8 px, and one-step flow on the two pairs below.

ANSWER (a) Above what shift does the one-step error grow quickly? (b) Why does one step come close on pair A (375 to 376) but badly undershoot pair B (300 to 301)?

TWO PAIRS, GIVEN TO YOU TASK2.IPYNB

PAIR A · frames 375 to 376
small motion (the hover). One-step flow comes close.

PAIR B · frames 300 to 301
large motion (mid-descent). One-step flow badly undershoots it.

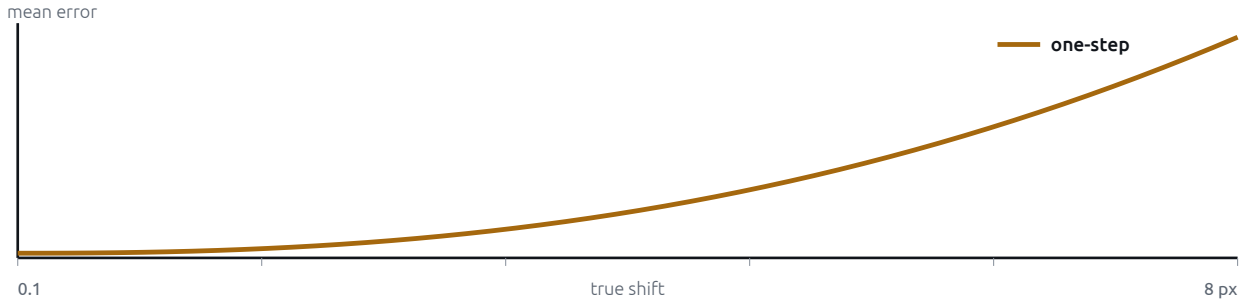


FIGURE 8 One-step flow error against the size of the true shift (illustrative shape: a single linearized update is only accurate for small motion).

STARTER-CODE INTERFACE · TASK2.IPYNB signatures only: the bodies are yours to write

```
# YOUR CODE cells of task2.ipynb   (bilinear_sample is provided)
def lucas_kanade_update(ix_window, iy_window, it_window): ...
    # A = rows [Ix, Iy], b = -It; np.linalg.solve(A.T @ A, A.T @ b); None if singular
def one_step_flow(image0, image1, point, window_radius=7, grad0=None): ...
    # np.mgrid window, bilinear_sample Ix, Iy and It on it, one update
```

DELIVERABLES TASK 2 · 30 MARKS

- ☐ `task2.ipynb`: your code, the checks run, your answers
- ☐ `outputs/task2/error_sweep.png`
- ☐ `outputs/task2/error_sweep.json`
- ☐ `outputs/task2/one_step_small_motion.png`
- ☐ `outputs/task2/one_step_large_motion.png`

MISSION CHECKPOINT

You now have:

gradients → one Lucas-Kanade step

TASK 03

TRACK FAST MOTION

Iterative and SIFT-guided tracking

30 MARKS

OBJECTIVE

Make Lucas-Kanade follow large motion: iterate it, track many points reliably, and start it from SIFT matches when the motion is too large.

STORY · COMMS LOG — INCOMING

The navigation officer reviews tracks that failed near touchdown.

› "Near touchdown some rocks jump further than a tracker can reach."
› "The archive recognises them by their signature (SIFT)."

REQUIRED WORK

3.1 Iterative Lucas-Kanade — 9 marks

Start from $(u, v) = (0, 0)$ or from a given initial guess, warp the second image with bilinear sampling, and repeat the Task 2 update until the increment is small or a maximum iteration count is reached. The provided cell adds your iterative curve to the error sweep of Task 2.3.

ANSWER (a) Up to what shift does the iterative tracker stay accurate?

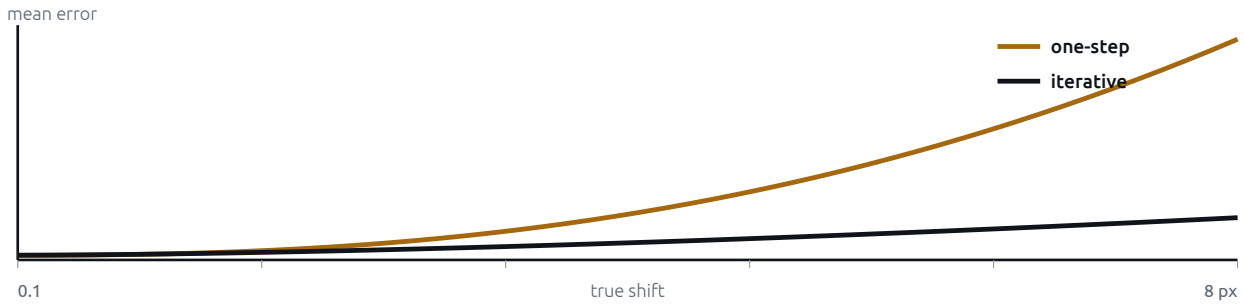


FIGURE 9 The same error sweep, with iterative Lucas-Kanade added: it stays accurate for a much larger shift (illustrative shape).

3.2 Track many points — 6 marks

Track many points at once and mark a track as failed when its window leaves either image or the least-squares system cannot be solved.

ANSWER (a) Can a track that is marked valid still be wrong? Give an example from your results.

3.3 Use SIFT to handle large motion

7 marks

PROVIDED HELPER · SIFT MATCHING PROVIDED/SIFT.PY

`track_with_sift` runs your tracker from (0, 0) and from a SIFT starting guess, and keeps the better fit:

```
tracked, valid = track_with_sift(track_points, image0, image1, points, sift_matches(image0, image1))
```

ANSWER (a) Why do iterative and SIFT-guided tracking give almost the same result on 375 to 376, but very different results on 300 to 301? Use the numbers of Task 3.4.

3.4 Compare the three trackers

8 marks

Compare one-step, iterative and SIFT-guided tracking on the two pairs of Task 2.3 and on a third pair near touchdown (409 to 410); the provided cells overlay all three trackers in one figure per pair.

ANSWER Fill in the table of mean displacements and failed-track rates (0 to 1). (a) Which tracker is right on each pair? (b) Why does SIFT-guided tracking lose many points at 409 to 410?



FIGURE 10 One-step tracking reaches only a fraction of a large true motion; iterating helps; a SIFT starting guess reaches almost all of it (illustrative).

STARTER-CODE INTERFACE · TASK3.IPYNB signatures only: the bodies are yours to write

```
# YOUR CODE cells of task3.ipynb
def track_point(image0, image1, point, window_radius=7, max_iterations=10,
                tolerance=1e-3, grad0=None, initial=None): ...
    # sample image1 at (xs + u, ys + v), add the update, repeat until it is tiny
def track_points(image0, image1, points, ..., initial=None): ...
    # gradients once, then track_point per point -> (tracked, valid)
```

DELIVERABLES TASK 3 · 30 MARKS

- ☐ `task3.ipynb`: your code, the checks run, your answers
- ☐ `outputs/task3/error_sweep.png`
- ☐ `outputs/task3/error_sweep.json`
- ☐ `outputs/task3/small_motion_compare.png`
- ☐ `outputs/task3/large_motion_compare.png`
- ☐ `outputs/task3/touchdown_compare.png`
- ☐ `outputs/task3/*_statistics.json` (small, large, touchdown; needed for grading)

MISSION CHECKPOINT

You now have:

one LK step → iterative tracking → SIFT-guided tracking

TASK 04

RECONSTRUCT THE FLIGHT

Connecting your trackers to the provided pipeline

20 MARKS

OBJECTIVE

Connect your trackers to the provided template code and recover the lander's velocity and 3D trajectory, then find out how the quality of your optical flow decides the result.

STORY · COMMS LOG — INCOMING

Mission control comes back on the line.

› "We have one altitude reading and the camera's focal length."

› "Then we can put the whole descent in metres. Fly it back."

PROVIDED: THE CAMERA MODEL AND THE FLIGHT CODE

The camera points straight down at flat ground. As the lander moves, the tracked ground features between two consecutive frames undergo a similarity transformation: they can scale, rotate and translate.

$$\mathbf{x}_{k+1} = s_k \, R(\theta_k) \, \mathbf{x}_k + \mathbf{t}_k \tag{4.1}$$

Using the tracked points from Tasks 1 to 3, `provided/task4/motion_model.py` estimates the scale s_k, rotation θ_k and translation $\mathbf{t}_k$ by least squares. The scale is related to the lander's altitude:

$$s_k = \frac{Z_k}{Z_{k+1}}. \tag{4.2}$$

A robust fitting step removes incorrectly tracked points and refits the motion.

Finally, `provided/task4/trajectory.py` converts the estimated motion over all frames into altitude, position, velocity and the lander's 3D trajectory, in metres and seconds. Neither file is yours to write, but you should be able to explain them: they are built entirely from your Task 1 to 3 tracks. Read them, run them through the notebook, and interpret what they produce.

Your goal: understand how simple 2D image motion can be used to recover the lander's motion through 3D space.

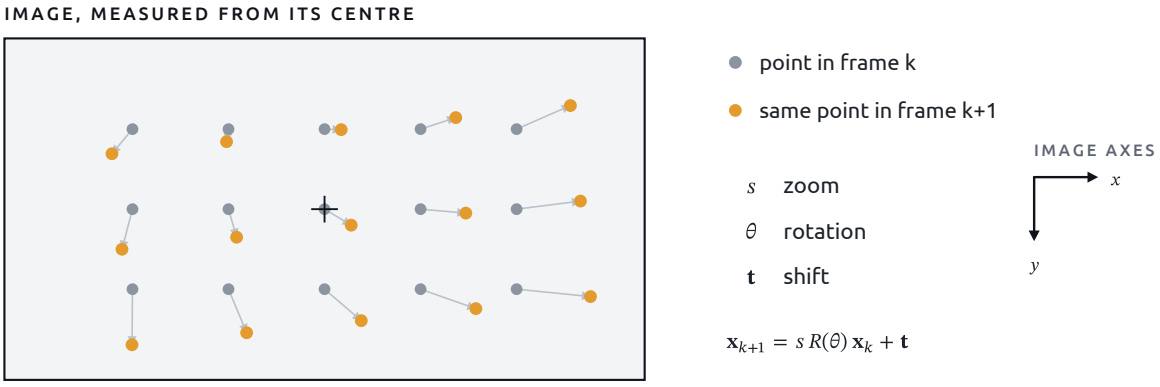


FIGURE 11 A similarity zooms, rotates and shifts the whole image (exaggerated here).

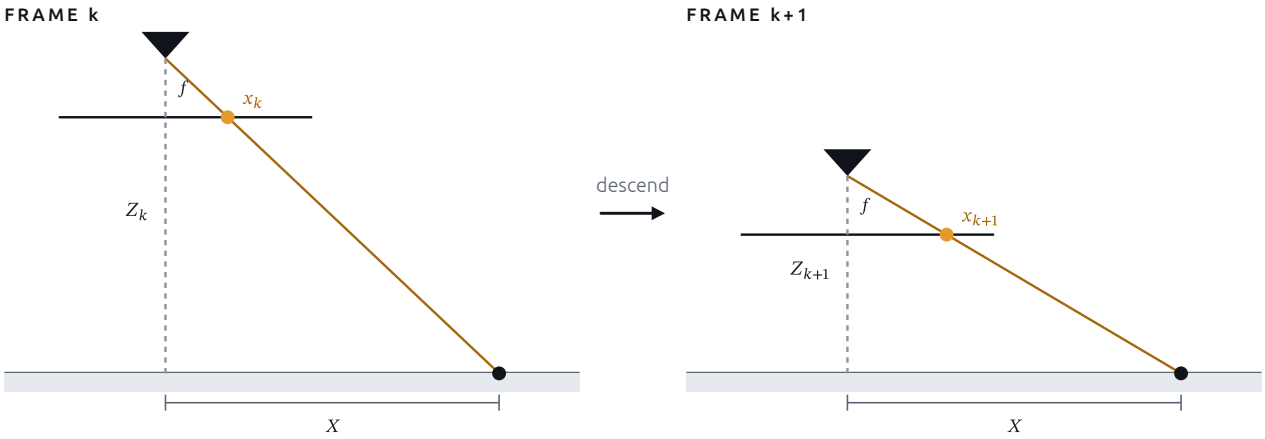


FIGURE 12 The same ground point seen from two altitudes: as the lander descends, the point moves away from the image centre.

REQUIRED WORK

4.1 Connect your trackers to the pipeline

4 marks

Fill in the Task 4.1 cell of `task4.ipynb`: the adapters `one_step`, `iterative` and `sift_guided`, which call your Task 2 and Task 3 code, and `track_pair`, which selects features (Task 1), runs a tracker and keeps the valid tracks. The tests check the wiring on synthetic data.

ANSWER (a) Why must `track_pair` drop the failed tracks before the motion fit?

STARTER-CODE INTERFACE · TASK4.IPYNB — signatures only: the bodies are yours to write

```
# the Task 4.1 YOUR CODE cell of task4.ipynb (your only code in Task 4)
def one_step(image0, image1, points): ...
    # track_points(image0, image1, points, TRACK_RADIUS, max_iterations=1)
def iterative(image0, image1, points): ...
    # the same with max_iterations=ITERATIONS
def sift_guided(image0, image1, points): ...
    # track_with_sift(track_points, ..., sift_matches(image0, image1), ...)
def track_pair(image0, image1, tracker): ...
    # select_features(image0, ...), run tracker, keep only the valid tracks
```

4.2 Read the motion from the images

4 marks

The provided cell tracks the whole descent with your SIFT-guided tracker; look at the zoom, rotation and shift plots.

ANSWER (a) Why is the zoom $s_k = Z_k / Z_{k+1}$? Use the pinhole model. (b) What are the phases of the descent, and at about which frames do they change? (c) Which pattern in these plots is not caused by the lander, and how can you tell?

4.3 Reconstruct the flight

4 marks

Look at the altitude, the velocity (v_X, v_Y, v_Z), the time to touchdown

$$\tau = \frac{Z}{-v_Z}, \tag{4.3}$$

the 3D trajectory and the ground track (seen from above, X to the right and Y up, Figure 13).

ANSWER Fill in the summary table. (a) Which result do you trust least, and why?

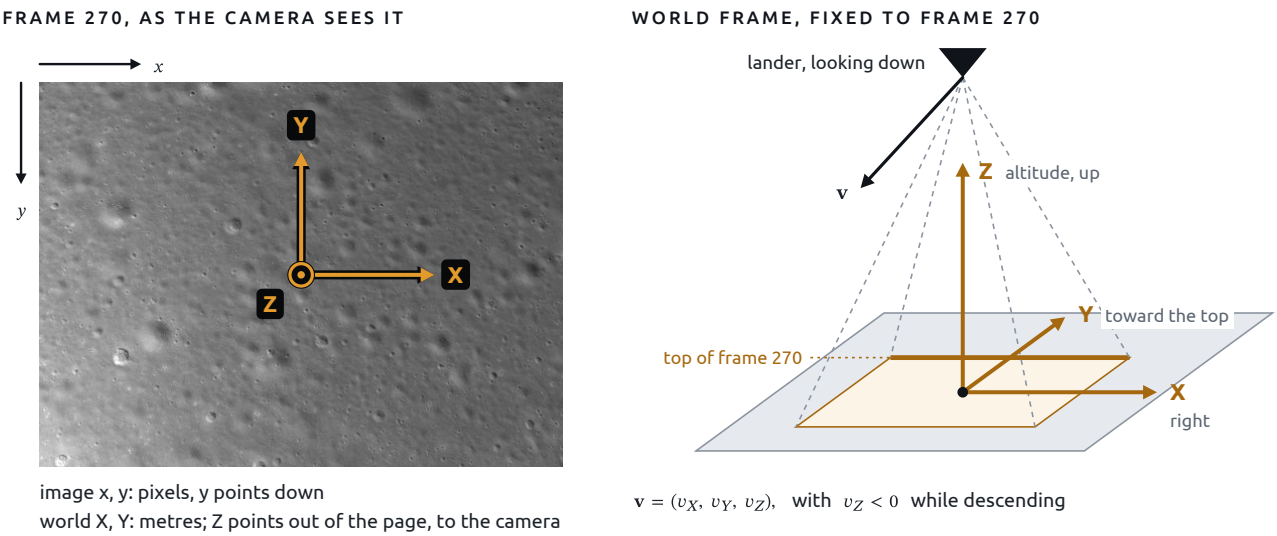


FIGURE 13 The axes of Task 4. Left: image coordinates x, y are pixels, with y pointing down; the world axes X and Y lie in the image plane and Z points out of the page, toward the camera. Right: the same right-handed world frame in 3D, fixed to frame 270.

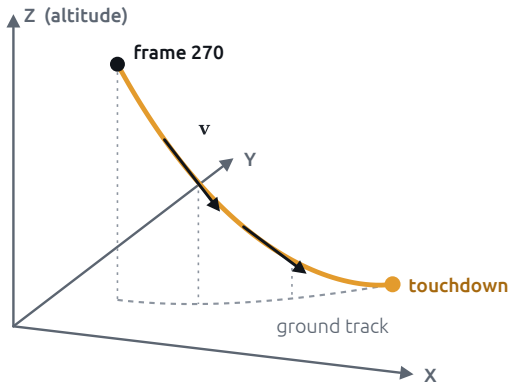


FIGURE 14 What Task 4 produces: the lander's position (X, Y, Z) frame by frame, and its ground track (an illustration, not data).

4.4 Compare the three trajectories

5 marks

The provided cell of Task 4.4 flies the pipeline once per tracker (one-step, iterative, SIFT-guided) on the same points and draws the three ground tracks together, seen from above.

ANSWER Fill in the table from `compare/summary.csv`. (a) Which trajectory do you trust, and which Task 3 result supports it? (b) Does a low median residual mean a tracker is right? Use your numbers.

PROVIDED CODE · PROVIDED/TASK4/COMPARE_TRACKERS.PY (SIMPLIFIED) run by the Task 4.4 cell

```
from provided.task4.run import fit_all_pairs, fly, motion_arrays   # provided
trackers = {"one-step": one_step, "iterative": iterative,  # your Task 4.1 functions
            "sift-guided": sift_guided}
for name, tracker in trackers.items():
    tracks = {pair: track_pair(image0, image1, tracker) for pair in consecutive_pairs}
    rows, _ = fit_all_pairs(tracks, centre)                          # provided
    flights[name] = fly(*motion_arrays(rows), constants, scale)    # provided

positions = {name: f["positions"] for name, f in flights.items()}  # [X, Y, Z]
plot_ground_tracks(positions, "outputs/task4/compare/ground_track.png")
```

An optional provided cell (`provided/task4/animate_trajectory.py`) turns the same three flights into an interactive 3D animation, `outputs/task4/trajectory_animation.html`, that you open in a web browser: play or scrub through the descent, rotate and zoom the scene, and hide trackers from the legend. It is not graded; it exists so you can watch, not just read, how far a tracker that cannot follow large motion drifts from the others.

4.5 Where does uncertainty come from?

3 marks

ANSWER (a) Why does a small error in each zoom grow along the descent? (b) What changes if the anchor altitude is 10 % too high? And if the focal length is?

DELIVERABLES TASK 4 · 20 MARKS

- ☐ `task4.ipynb`: your code, the check run, your answers and the Final Mission Report
- ☐ `outputs/task4/zoom.png`
- ☐ `outputs/task4/rotation.png`
- ☐ `outputs/task4/shift.png`
- ☐ `outputs/task4/altitude.png`
- ☐ `outputs/task4/velocity.png`
- ☐ `outputs/task4/time_to_touchdown.png`
- ☐ `outputs/task4/trajectory_3d.png`
- ☐ `outputs/task4/ground_track.png`
- ☐ `outputs/task4/summary.json`
- ☐ `outputs/task4/compare/ground_track.png`
- ☐ `outputs/task4/compare/summary.csv`

MISSION CHECKPOINT

You now have:

your optical flow → motion → trajectory → which to trust

ES666 · ASSIGNMENT 3

MISSION DEBRIEF

Final mission report · Marking scheme · Working instructions

100 MARKS

FINAL MISSION REPORT · BONUS, 5 MARKS

WHAT DID WE LEARN?

Answer at the end of `task4.ipynb`, in one or two sentences each, with one piece of evidence from your results.

Q1 Where in the descent does “all pixels in a window move together” fail most, and what in your results shows it?

Q2 Would a pyramidal (coarse-to-fine) KLT tracker fix 300 to 301? Would it fix 409 to 410? Why?

Q3 If you ignored the pattern from Task 4.2, which Task 4.3 results would be wrong, and by how much?

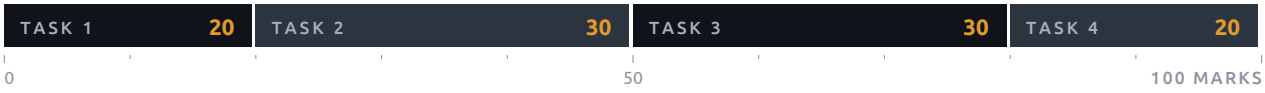
Q4 Where does the “flat ground, camera straight down” model fail, and what error does it cause?

Q5 *Name one independent measurement that would check your trajectory, and what it would check.*

DELIVERABLES AND MARKS

MARKING SCHEME

TASK	TOPIC	WEIGHT	MARKS
TASK 1	Gradients, motion constraint, trackable points		20
TASK 2	Lucas-Kanade, one step		30
TASK 3	Iterative + SIFT-guided tracking		30
TASK 4	Tracker wiring + trajectory interpretation		20
	TOTAL		**100**



Bonus: the Final Mission Report (the five questions after Task 4) is worth up to **5 bonus marks** on top of the 100 above, for tying your results across all four tasks together.

READ THIS BEFORE YOU START

WORKING INSTRUCTIONS

01 Read `README.md` for setup and how the notebooks work, and this handout for what each task asks you to implement and interpret.

02 Work in `task1.ipynb` to `task4.ipynb`, in order. Write your code only in the cells whose first line starts with `# YOUR CODE`: only those are graded, read from the saved notebook. Add your own cells for experiments and notes as you like. Every other file is provided and must not be changed.

03 Verify as you go: each `check(...)` cell tests your functions (`python -m pytest provided/taskN` runs the same tests on the saved notebook), and the provided cells show your results on the real frames and save them in `outputs/`.

04 Before you submit, run everything from a clean start with `python run_notebooks.py` (or restart the kernel and run all cells of each notebook), and check that it finishes without errors.

05 Fill in your **name and roll number** at the top of `task1.ipynb`, then run `python make_submission.py`: it builds `report.html` and writes `submission.zip` (the notebooks, the report and the `outputs/` files used for grading) and warns about anything still missing or TODO; you can still submit a partial report.

06 The 3D animation cell at the end of Task 4.4 is optional and not graded: it writes `outputs/task4/trajectory_animation.html`, to open in a browser. Watch it for intuition; do not write it up.

07 Questions: ask on Google Classroom first, so the rest of the class sees the answer too; email `shekhar.hement@iitgn.ac.in` if it needs to stay private.



🤖 **Yes, AI can solve this assignment in one prompt.**

Your trajectory will be reconstructed. Your understanding will not.

END OF MISSION BRIEF

Touchdown frame of the descent sequence.

IMAGE · SEQUENCE FRAME 413